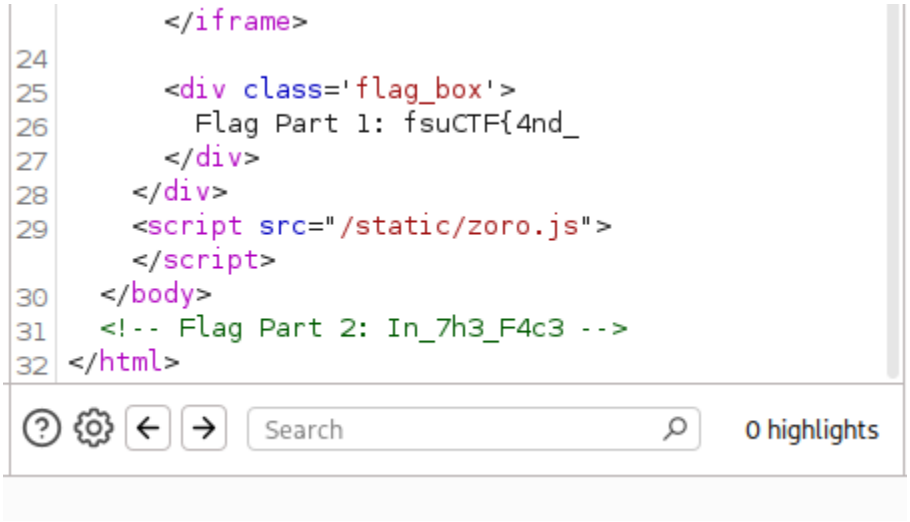
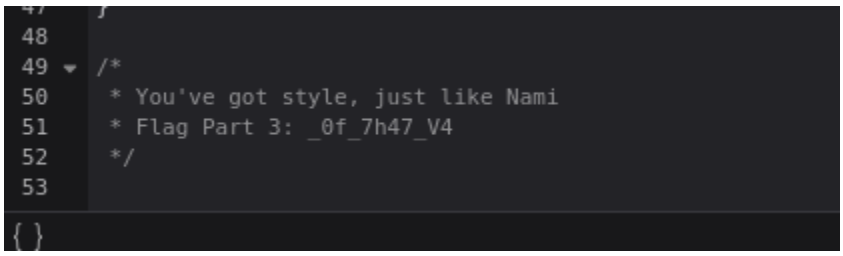
Problem ID	Captured Flags	Steps
The One Piece	fsuCTF{4nd_In_7h3_F4c3_0f_7h47_V457_7r345ur3_Which_W43_Which_W41_Ind33d_H3_14u6h3d}	1. Open Burp and explore the page 2. Scavenge for the pieces of the flag in the html, headers, css, js and robots.txt file 3. Change the pirate cookies on the /laughtale page to Gol D Roger to get the last piece
ai_training	fsuCTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}	1. Pass through the WIP 2. Know that is a Flask inside of a docker 3. Search for the app.py file using the image vulnerability
web_books	fsuCTF{5_s7ar5_t0_7h3_b0t_5up3r_n1c3!}	1. Identified XSS vulnerability 2. Attacked using a webhook stript 3. Got the Librarians cookies 4. Navigated to the admin page
NERV-HQ	fsuCTF{g37_in_7h3_c7f_m4chine_5hinji}	1. Knew it was an SQLi problem 2. Assumed the number of columns was the same in all the tables 3. Tried numerous injections using obfuscation to bypass the WAF and figured out the blacklist 4. Tried Boolean Blind (didnt work) 5. Rewatched class videos and realized I need to get the name of the database first 6. Recreated the payloads from class with the blacklist applied

The One Piece

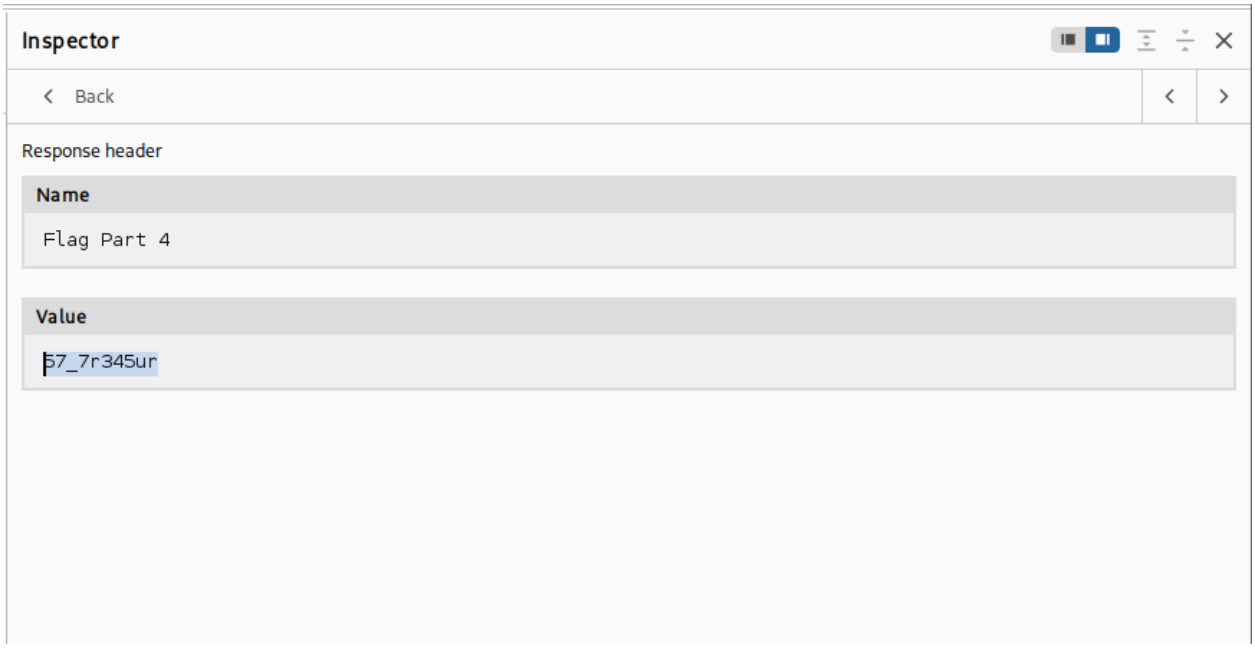
I open the webpage in burp and find the first two parts in the HTML:



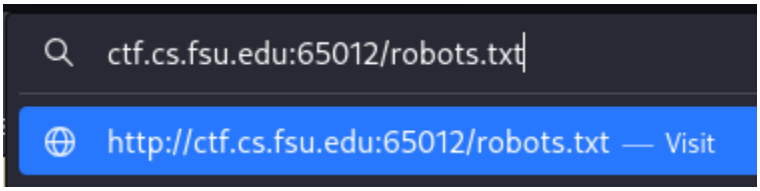
I then navigate over to the css file in inspect mode and check there for anything:



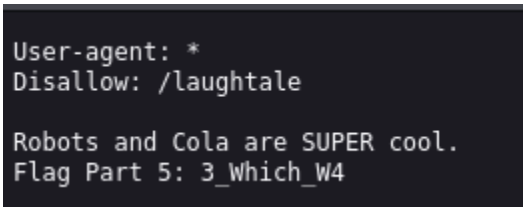
Then I navigate over the response headers in burp to find the fourth piece of the flag:



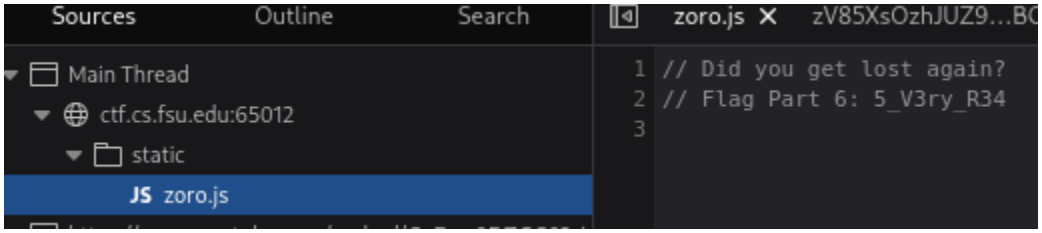
I then check if the website has a robots.txt webpage:



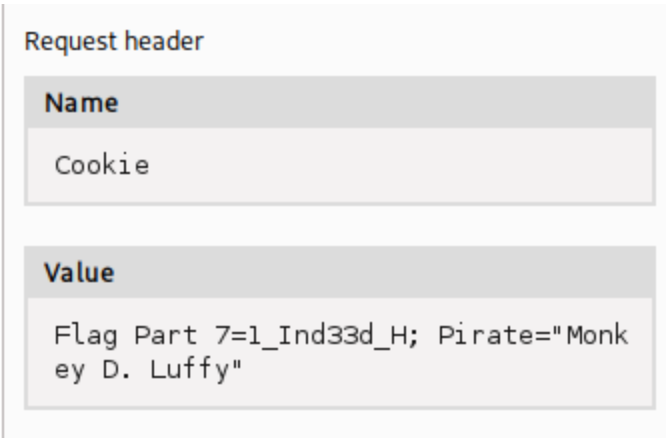
Which it does and gives me the next part of the flag:



I then check the zoro.js file in the debugger section of the website on inspect:



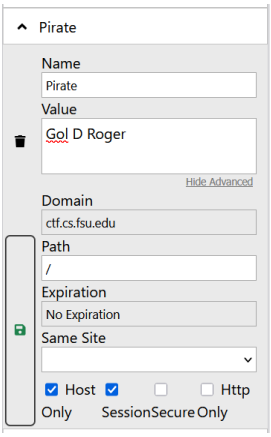
I then move over to the cookies via burp and find the next piece:



And from part 5 we know there is a webpage called ‘laughtale’ so I navigate over there:

Only the King of the Pirates made it to Laughtale!

The king of the pirates is Gol D Roger, so I changed the cookie ‘pirates’ from Monkey D Luffy to Gol D Roger.

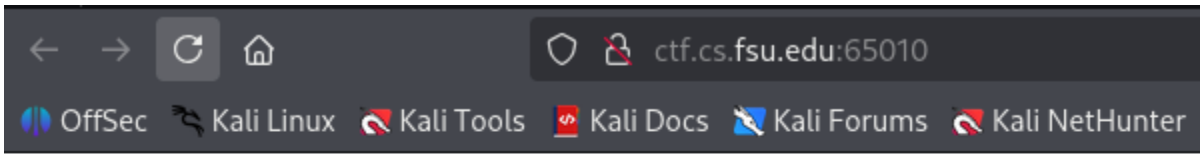


Flag Part 8: 3_14u6h3d}

Flag:

fsuCTF{4nd_In_7h3_F4c3_0f_7h47_V457_7r345ur3_Which_W43_Which_W41_Ind33d_H3_14u6h3d}

AI_Training

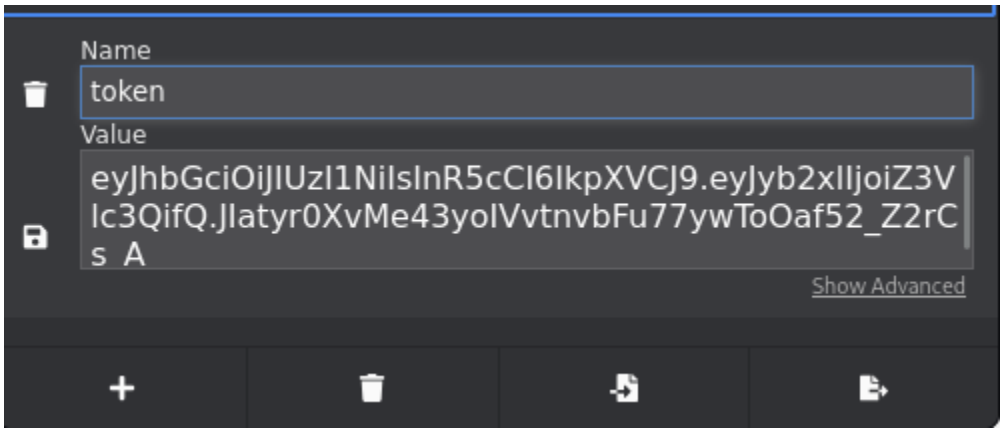


AI_Training: WIP

Note: Administrators only

Enter Training Portal

We start with the following webpage. I know this is a WIP vulnerability so I check the cookies.

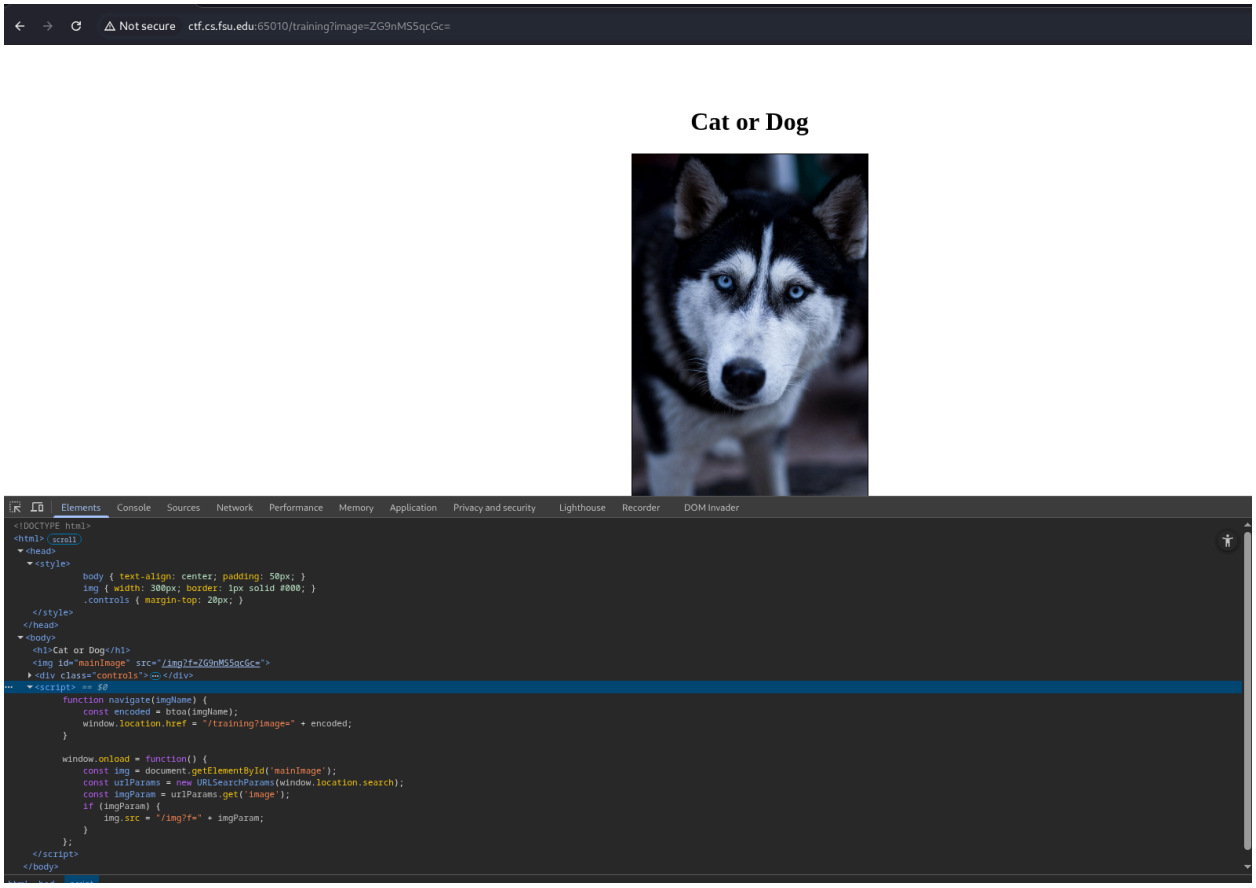


This is clearly a JWT so I go to the JWT debugger so that I can change my privilege to admin:

So I change it to:

And I paste into the cookie token:

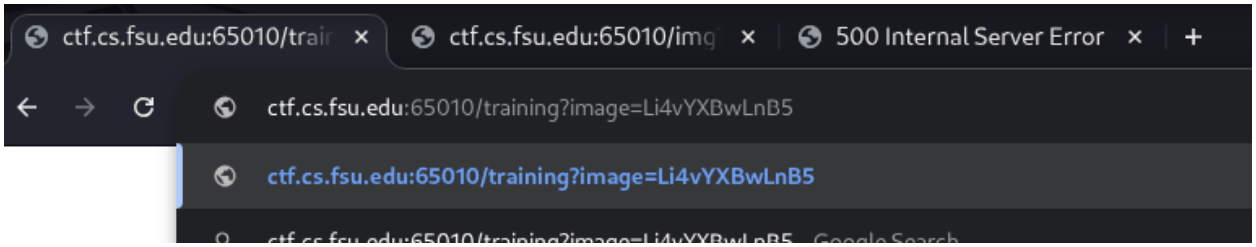
And then I refresh the page and click “Enter Training Portal”



I am met with this page that “trains” an AI on a cat and dog. In reality it just has a base64 encoded image changer. So everytime I click the button it swaps from one image to the next. The images are base64 encoded so I can grab the app.py file in local directory on the docker (as provided in the hint). I try many types of inputs in the url before I came to this conclusion such as: ../../../flag.txt, main.py, app.py. Until I realized the format is probably I am inside of the folder with the images so I need to back out with the ..

and go to the app.py. So my final string ended being ../app.py . I base64 encoded it which is:

Li4vYXBwLnB5 and pasted it in the url under ?image=.....



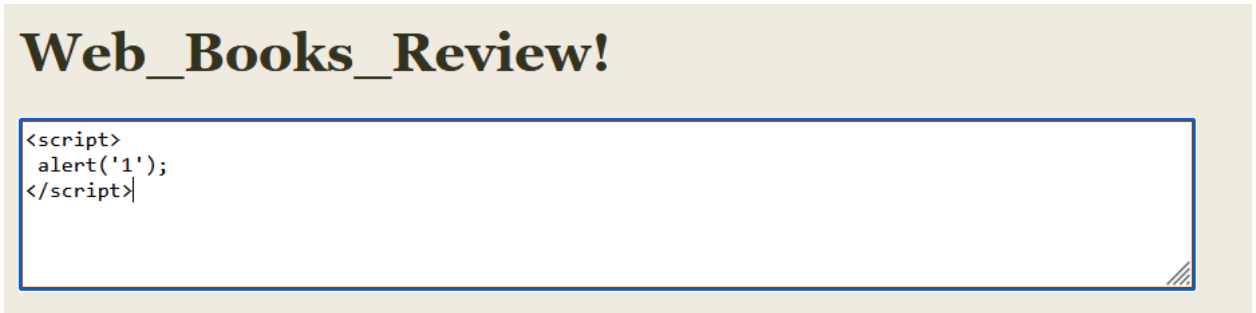
Which gave me:

So I right click on the image and click open in new tab:

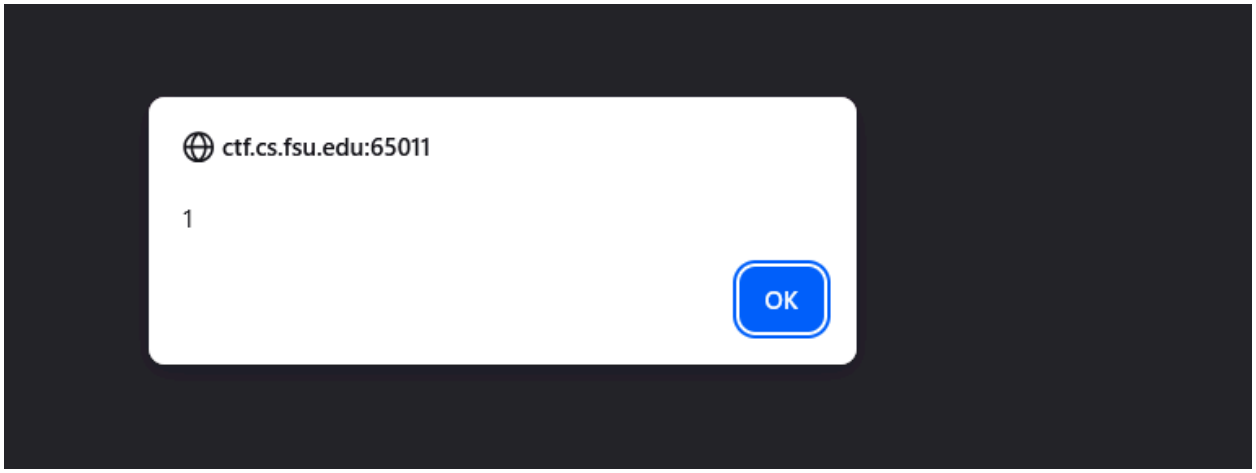
```
from flask import Flask, render_template, request, make_response, send_file
import jwt
import os
import base64

app = Flask(__name__)
app.config['SECRET_KEY'] = 'fsuCTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}'
```

Flag: fsuCTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}

web_books

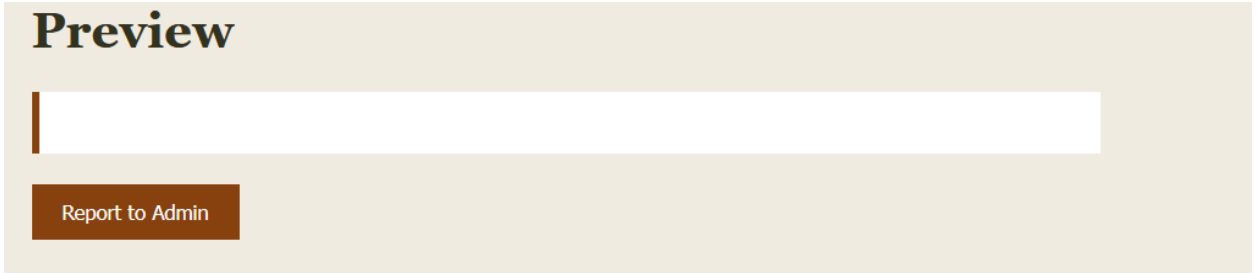


I first test if this is a XSS vulnerability:



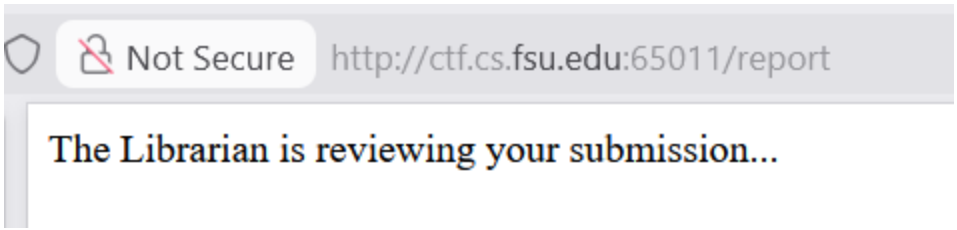
I find that it is vulnerable so naturally I gravitate towards a webhook because that is what we did in class. I craft my payload:

And send it:

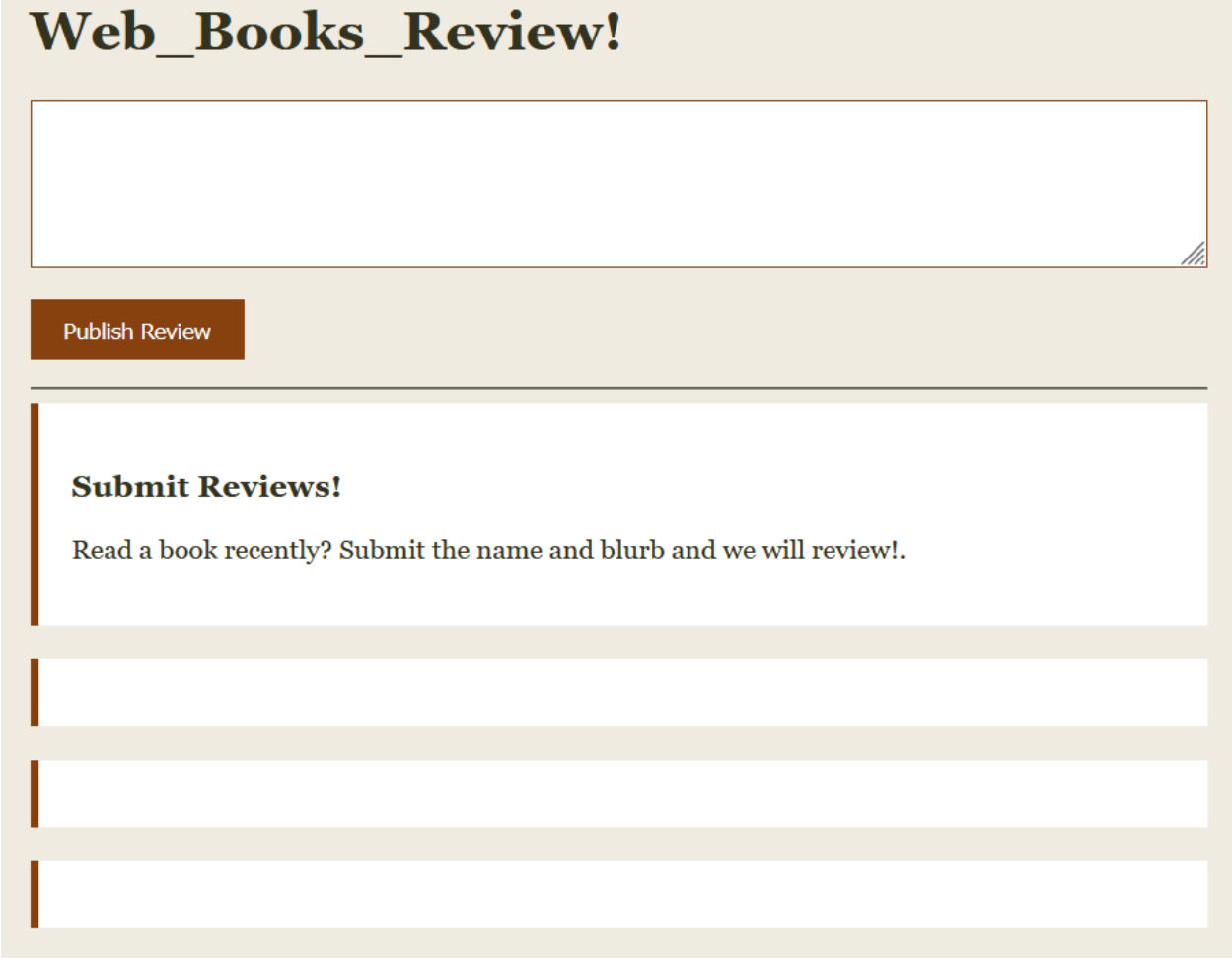


The code is executed because we are given nothing in the output where usually, if we inputted text, it would be review by the author there.

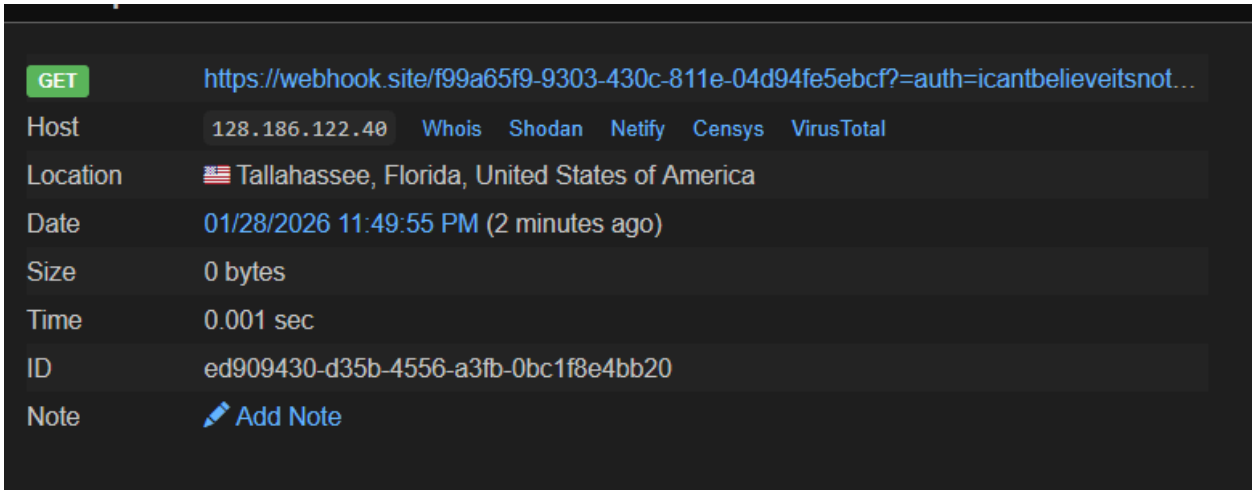
I then click report to admin:



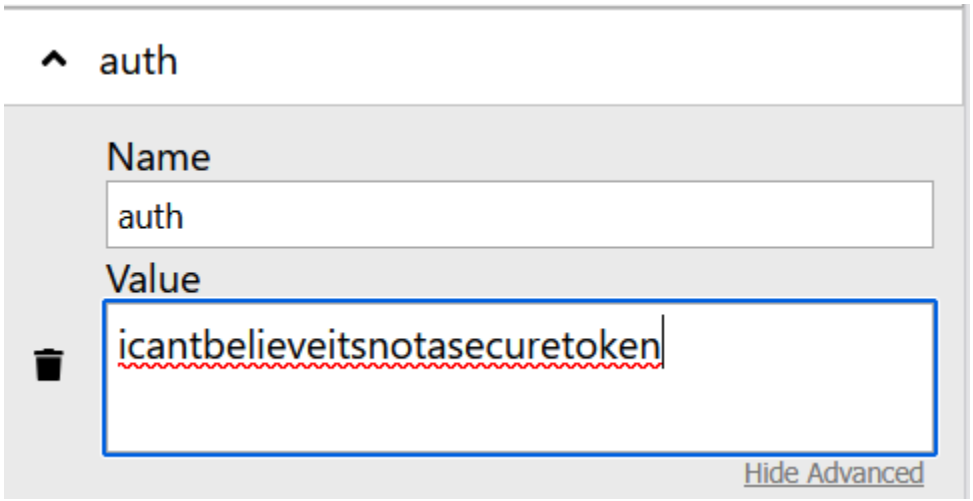
It takes me to the report page and says it is reviewing my submission. Little does it know I just stole its cookies!



I navigate back to the homepage and see that all my scripts have been sent.



And from webhook.site we get the auth=icantbelieveitsnotasecuretoken.

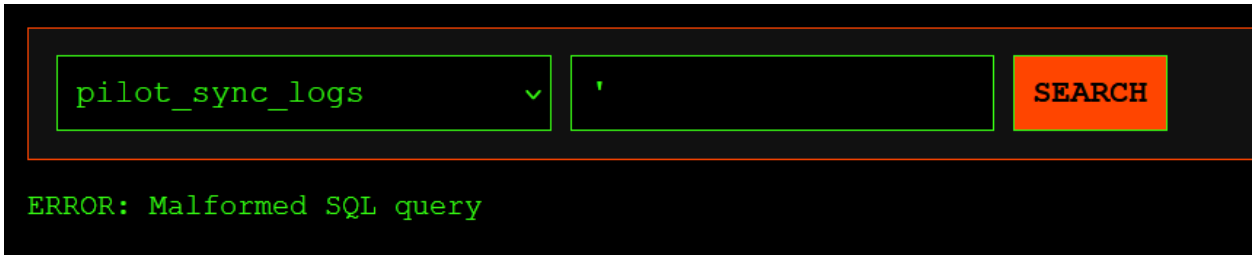


I use that for the cookie, save it and reload the page. And nothing happened. I checked the HTML, robots.txt and every page. I asked Gemini what hidden pages there might be and it said /admin. So I navigated there with the auth=icantbelieveitsnotasecuretoken cookie, and…

Flag: fsuCTF{5_s7ar5_t0_7h3_b0t_5up3r_n1c3!}

NERV-HQ

I started the problem by inputting multiple simple SQL statements and got 2 alarming outputs:

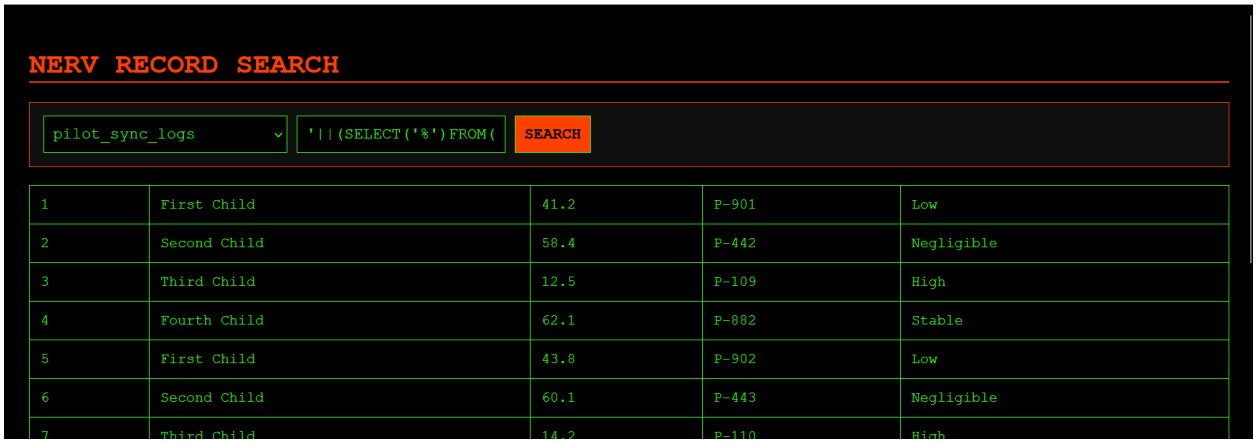


These outputs mean that there is a blacklist of what SQL statements can actually be inputted. I tried loads of means to bypass this filter and figured out that my payload needed two things:

1. No spaces
2. Wrapped in single quotes
3. Cannot end with --

With those conditions in mind I first tried boolean blind to “guess” the name of the table which the payload look something like this:

'||(SELECT('%')FROM(sqlite_master)WHERE(tbl_name)LIKE('in_t%')AND(sql)LIKE('%secret%'))||'



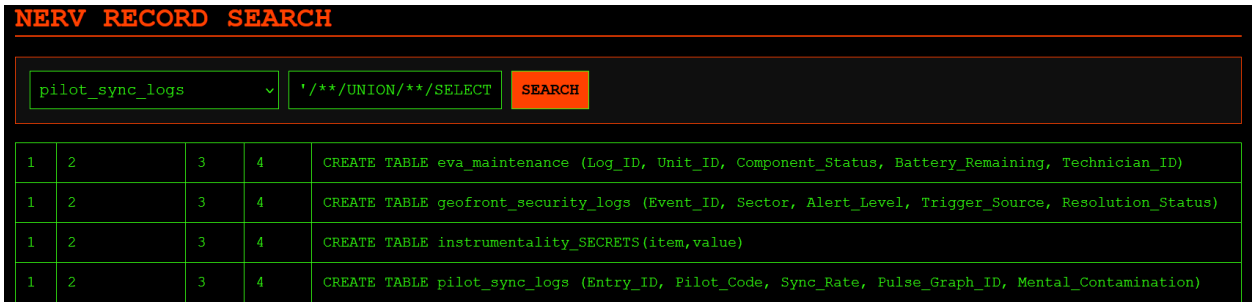
Basically it would show the table if I “guessed” the name of the database correctly. This was very tedious. I rerouted by approach to try to find the name of the database using the SQL we used in class. We used:

'UNION SELECT 1,sql FROM sqlite_schema--

This statement successfully displays the name of the databases if the number of columns align correctly. From this I started to develop the SQLi using the blacklist obfustication and got:

'//UNION/**/SELECT/**/1,2,3,4,sql/**/FROM/**/sqlite_schema/**/WHERE/**/'1'!='2**

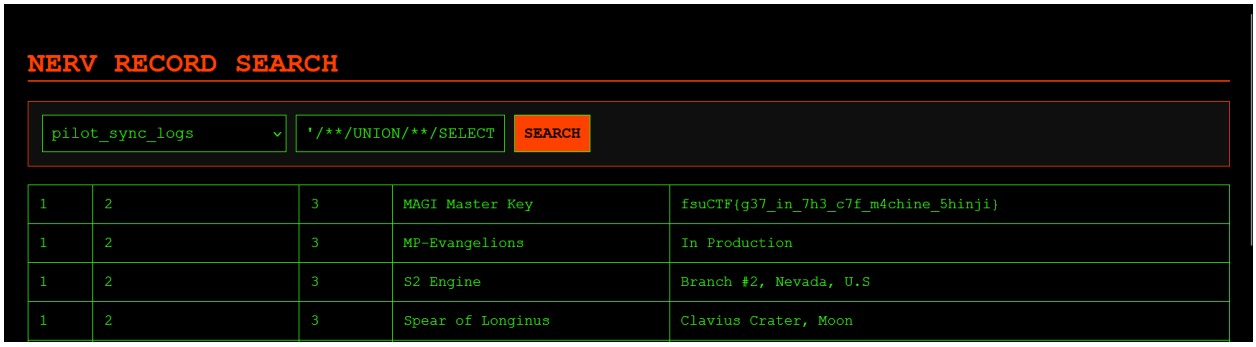
I found how to bypass the blacklist with the no spaces rule with // from PortSwigger.My next issue was how to wrap the payload in the single quotes to get rid of the ERROR: Malformed and NOT use - -. I used gemini to come up with the TRUE statement **WHERE/**/‘1’!=2** that absorbs the trailing garbage without using the banned `--` comment. Which gave the following output:



So now I know the database is called intrumentality_SECRETS and holds tables item and value. In order to extract this data we need to hold the 5 column rule. So I construct my payload the same way as the last one with the blacklist obfustication:

'//UNION/**/SELECT/**/1,2,3,item,value/**/FROM/**/instrumentality_SECRETS/**/WHERE/**/'1'!='2**

Which gave me the flag:



Flag: fsuCTF{g37_in_7h3_c7f_m4chine_5hinji}